AI Job Search

Instalación y uso paso a paso del repositorio MadsLorentzen/ai-job-search, con todos los comandos y el contenido de cada carpeta.

Basada en el README, el SETUP.md y el documents/README.md del repositorio, leídos en octubre de 2026. El proyecto cambia con frecuencia: si un comando falla, compara con la versión actual en GitHub. Es un proyecto independiente y no está afiliado a Anthropic.

1. Qué es y cómo funciona

AI Job Search es un framework que convierte Claude Code en un asistente para buscar empleo. Tú rellenas tu perfil una vez y después le pasas ofertas: evalúa si encajas, adapta tu CV y tu carta de presentación en LaTeX, hace que un segundo agente revise el borrador y compila los PDF finales. Todo se ejecuta en tu ordenador, dentro de la carpeta del repositorio.

Comando	Qué hace
`/setup`	Entrevista inicial o lectura de tus documentos para crear tu perfil.
`/scrape`	Busca ofertas en varios portales, quita duplicados y las ordena por encaje.
`/rank`	Puntúa en lote las ofertas encontradas y devuelve una lista corta.
`/apply <url>`	Evalúa la oferta, redacta CV y carta, las revisa con otro agente y compila los PDF.
`/interview`	Prepara una entrevista concreta con preguntas probables y simulacro.
`/outcome`	Registra qué pasó con cada candidatura y archiva los documentos enviados.

Importante sobre los portales. Los cuatro buscadores incluidos (Jobindex, Jobnet, Jobbank y Jobdanmark) son de Dinamarca. Solo **linkedin-search** y **freehire-search** sirven para cualquier país. Para tu mercado puedes generar un buscador propio con `/add-portal`.

2. Requisitos

Herramienta	Para qué sirve	Comprobación
Claude Code (CLI)	Es el motor de todo. Necesita una suscripción Claude Pro, Max o Team, o créditos de la API de Anthropic. No tiene capa gratuita.	`claude --version`
Node.js y npm	Hacen falta para el comando de instalación de Claude Code.	`node --version`
Git y GitHub CLI	Para clonar el repositorio y trabajar con tu copia.	`git --version` `gh --version`
Python 3.10+	Herramienta de salarios y comprobación ATS.	`python3 --version` `py --version (Windows)`
Bun	Ejecuta los buscadores de empleo, escritos en TypeScript.	`bun --version`
LaTeX	Compila el CV con lualatex y la carta con xelatex.	`lualatex --version` `xelatex --version`
pypdf (opcional)	Extrae el texto del CV para la comprobación ATS.	`pip install pypdf`

3. Instalación paso a paso

Paso 1. Instalar Claude Code

```
npm install -g @anthropic-ai/claude-code
```

La primera vez que ejecutes `claude` te pedirá iniciar sesión con tu cuenta de Claude o con una clave de API.

Paso 2. Comprobar Python

```
python3 --version     # necesitas 3.10 o superior
py --version          # alternativa en Windows
```

Si tu sistema expone Python como `python` en vez de `python3`, usa ese nombre en todos los comandos de la guía.

Paso 3. Instalar Bun

```
# macOS y Linux
curl -fsSL https://bun.sh/install | bash

# Windows (PowerShell)
powershell -ExecutionPolicy Bypass -c "irm https://bun.sh/install.ps1 | iex"

# Windows con winget
winget install Oven-sh.Bun
```

Paso 4. Instalar LaTeX

Con una distribución completa funciona todo sin más ajustes.

Sistema	Distribución
Windows	MiKTeX: https://miktex.org/download
macOS	MacTeX: https://tug.org/mactex/
Linux (Debian/Ubuntu)	`sudo apt install texlive-full`
Linux (Fedora)	`sudo dnf install texlive-scheme-full`

El CV se compila con **lualatex** porque pdflatex suele fallar en MiKTeX con errores de *fontawesome5*. La carta usa **xelatex** porque su clase `cover.cls` necesita *fontspec* para las fuentes Lato y Raleway.

Si prefieres una instalación mínima (TinyTeX o BasicTeX en macOS)

```
curl -fsSL https://yihui.org/tinytex/install-bin-unix.sh -o /tmp/tinytex-install-bin-unix.sh
sh /tmp/tinytex-install-bin-unix.sh /tmp --no-path
export PATH="$HOME/Library/TinyTeX/bin/universal-darwin:$PATH"

tlmgr install \
  moderncv fontawesome5 fontawesome6 academicons import luatexbase pgf \
  titlesec textpos xltxtra xunicode cite realscripts needspace
```

Con BasicTeX o MacTeX, añade antes `/Library/TeX/texbin` al PATH y ejecuta el mismo `tlmgr install`.

Si usas Basic MiKTeX en Windows

Por defecto MiKTeX abre una ventana para cada paquete que falta, y eso bloquea las terminales no interactivas como la de Claude Code. Activa la instalación silenciosa:

```
initexmf --admin --set-config-value=[MPM]AutoInstall=1
initexmf --set-config-value=[MPM]AutoInstall=1
```

La primera línea es para instalaciones de todos los usuarios (PowerShell como administrador) y la segunda para instalaciones de un solo usuario. Ejecutar las dos no causa problemas. Si prefieres preinstalar los paquetes:

```
mpm --admin --install=moderncv --install=fontawesome5 --install=fontawesome6 `
  --install=academicons --install=import --install=luatexbase --install=pgf `
  --install=titlesec --install=textpos --install=xltxtra --install=xunicode `
  --install=cite --install=realscripts --install=needspace
```

Quita `--admin` si MiKTeX está instalado solo para tu usuario.

Comprobación rápida de LaTeX (una vez clonado el repositorio)

```
cd cv && lualatex -interaction=nonstopmode -halt-on-error main_example.tex && cd ..
```

Si genera `main_example.pdf` sin errores, el CV ya compila. SETUP.md incluye además una prueba equivalente para la carta con xelatex.

Paso 5 (opcional). Extractor de texto para la comprobación ATS

/apply extrae el texto del PDF del CV y comprueba que un sistema ATS lo lee bien: datos de contacto, orden de lectura y palabras clave. El extractor por defecto es pypdf; Poppler es la alternativa. Si no tienes ninguno, esa comprobación se sustituye por una revisión visual.

```
pip install pypdf

# Alternativa: Poppler (pdftotext)
brew install poppler               # macOS
sudo apt install poppler-utils     # Debian/Ubuntu
choco install poppler              # Windows
```

Paso 6. Obtener el repositorio

> **Un fork de GitHub siempre es público.** GitHub no permite forks privados de repositorios públicos, y **/setup** escribe tus datos personales (nombre, contacto, historial laboral, expectativas salariales) en ficheros que Git rastrea. Si lo vas a usar para tu propia búsqueda, no subas esos cambios a un fork: usa un repositorio privado y deja este repo como **upstream**. Haz un fork solo si vas a contribuir al proyecto.

Opción A. Repositorio privado (recomendada para uso personal). Esta secuencia es una adaptación mía de la receta que describe SETUP.md en la sección 8; el repo no la da literal.

```
git clone https://github.com/MadsLorentzen/ai-job-search.git mi-busqueda-empleo
cd mi-busqueda-empleo
git remote rename origin upstream
gh repo create mi-busqueda-empleo --private --source=. --remote=origin
git push -u origin master
```

Opción B. Fork (solo para contribuir). Esta es la secuencia oficial de SETUP.md. La última línea no es opcional: sin ella, un `gh issue create` posterior se publicaría en el repositorio original, con tu usuario y sin que puedas borrarlo.

```
gh repo fork MadsLorentzen/ai-job-search --clone
cd ai-job-search
gh repo set-default <tu-usuario-github>/ai-job-search
```

Paso 7. Instalar las dependencias de los buscadores

Ejecuta esto desde la raíz del repositorio. Para linkedin-search y freehire-search la instalación es opcional: no tienen dependencias de ejecución y solo descargan tipos de TypeScript.

```
# Bash / zsh / Git Bash
for tool in jobbank-search jobdanmark-search jobindex-search jobnet-search \
            linkedin-search freehire-search; do
  (cd .agents/skills/$tool/cli && bun install)
done
```

```
# PowerShell
$tools = @("jobbank-search", "jobdanmark-search", "jobindex-search",
           "jobnet-search", "linkedin-search", "freehire-search")
foreach ($tool in $tools) {
  Push-Location ".agents/skills/$tool/cli"
  bun install
  Pop-Location
}
```

Paso 8. Arrancar Claude Code en la carpeta

```
claude
```

A partir de aquí los comandos que empiezan por `/` se escriben dentro de Claude Code, no en la terminal normal.

4. Crear tu perfil con /setup

```
/setup
```

Claude detecta qué tienes y te ofrece tres caminos. Los tres terminan con los mismos ficheros de perfil rellenos.

Camino	Cuándo usarlo	Cómo funciona
A. Carpeta documents/	Tienes varios documentos (CV, LinkedIn, títulos, cartas).	Lee todo lo que hay en documents/, lo cruza y te propone el perfil.
B. Un solo CV	Tienes un único CV.	Lo mencionas con @ o pegas el texto; Claude lo extrae y pregunta lo que falte.
C. Entrevista	No tienes documentos a mano.	Respondes preguntas estructuradas sección a sección.

Cuanto más detalle pongas, mejores salen las candidaturas. Describe lo que hiciste en cada puesto, con proyectos, herramientas y resultados medibles, y no solo el título. Con "Python" a secas el sistema tiene poco material; con "construí pipelines de predicción de bajas en Python con scikit-learn" puede adaptar tu experiencia a cada oferta.

Qué ficheros rellena /setup

Fichero	Contenido
`CLAUDE.md`	Tu perfil de candidato completo y las reglas del flujo.
`01-candidate-profile.md`	Formación, experiencia y habilidades, estructuradas.
`02-behavioral-profile.md`	Perfil de comportamiento (evaluación o autoevaluación).
`04-job-evaluation.md`	Áreas de encaje, objetivos de carrera y criterios para puntuar ofertas.
`05-cv-templates.md`	Plantillas de la frase de perfil del CV según el tipo de puesto.
`07-interview-prep.md`	Ejemplos STAR sacados de tu experiencia.
`cv/main_example.tex`	Tu CV en LaTeX con tus datos reales.
`search-queries.md`	Consultas que usa /scrape. El README no indica su ruta; localízalo con `find . -name search-queries.md`

Los ficheros 01, 02, 04, 05 y 07 viven en `.claude/skills/job-application-assistant/`.

Volver a ejecutar partes del perfil

```
/setup --section skills
/setup --section experience
/setup --section search
```

La sección `search` es la más útil con el tiempo: repite la entrevista de búsqueda y te sugiere tipos de puesto en los que no habías pensado.

5. La carpeta documents/ en detalle

Aquí va tu material de partida. **/setup** lo lee todo y puedes volver a ejecutarlo cuando añadas documentos: combina lo nuevo con lo existente y nunca sobrescribe nada sin preguntarte. Esta carpeta está excluida de Git, así que no se sube aunque hagas push.

```
documents/
|-- cv/                     # tu CV maestro (PDF o LaTeX)
|-- linkedin/               # exportación de tu perfil de LinkedIn (PDF)
|-- diplomas/               # títulos y expedientes
|-- references/             # cartas de recomendación
|-- applications/           # candidaturas anteriores
|   `-- <empresa>_<puesto>/
|       |-- job_posting.md   # texto de la oferta original
|       |-- cover_letter.tex # la carta que enviaste
|       |-- cv_draft.tex     # la variante de CV que enviaste
|       `-- outcome.md       # resultado y notas
`-- README.md
```

Carpeta	Qué poner	Formatos	Qué extrae /setup
`cv/`	Tu CV maestro: la versión más completa, sin adaptar a ninguna oferta. Los CV adaptados los genera /apply. Si hay varios, los cruza para comprobar coherencia.	.pdf, .tex	Puestos, empresas, fechas y logros; formación; habilidades técnicas; premios y publicaciones; contacto.
`linkedin/`	Tu perfil exportado. En LinkedIn: perfil, Más, Guardar en PDF. Solo se espera una exportación; si hay varias, usa la más reciente.	.pdf	Experiencia y fechas (las cruza con el CV), habilidades, certificaciones, voluntariado, publicaciones, resumen y recomendaciones recibidas.
`diplomas/`	Títulos y expedientes. Usa nombres descriptivos, por ejemplo `msc_fisica_ucph_2025.pdf`; el nombre no afecta a la lectura.	.pdf	Nombre oficial del título, fecha de graduación, notas si se ven e institución.
`references/`	Cartas de antiguos jefes o colaboradores, por ejemplo `reference_ole_frandsen.pdf`.	.pdf, .txt, .md	Nombre, cargo y organización del referente; citas y valoraciones; lenguaje de competencias.
`applications/`	Una subcarpeta por candidatura, con nombre en minúsculas y guiones bajos: `acme_ml_engineer`.	.md, .tex	Qué roles te llevan a entrevistas, estilo de tus cartas y CV, y señales para calibrar la evaluación.

Los cuatro ficheros de cada candidatura

- **job_posting.md**: pega aquí el texto completo de la oferta. Sirve para deducir qué tipos de puesto buscas y calibrar `04-job-evaluation.md`.
- **cover_letter.tex**: la carta que enviaste. De ella se extraen patrones de estilo y estructura para `06-cover-letter-templates.md`.
- **cv_draft.tex**: la variante de CV que enviaste. Alimenta los estilos de frase de perfil de `05-cv-templates.md`.
- **outcome.md**: se rellena cuando la candidatura se resuelve. Puedes mantener todo esto a mano o dejar que lo haga `/outcome`.

Formato de outcome.md

```
# Outcome: <Empresa> - <Puesto>

**Status:** in_progress | hired | offer_declined | rejected | no_response | interview_only

**Date resolved:** YYYY-MM-DD

## Interview stages reached
- [ ] Phone screen
- [ ] Technical interview
- [ ] Case interview
- [ ] Final round
- [ ] Offer received

## Notes
Qué pasó, qué feedback recibiste, qué harías distinto y qué parecía valorar la empresa.
```

El estado `in_progress` marca una candidatura abierta. La calibración de /setup solo saca conclusiones de las que ya tienen un estado final. Si ejecutas `/interview`, cada etapa deja un fichero `interview_prep_<etapa>.md` en la misma carpeta; /setup lo ignora.

Qué formatos lee /setup

Formato	¿Lo lee?	Nota
.pdf	Sí	Se lee directamente.
.tex	Sí	Estructura y contenido.
.md y .txt	Sí	Texto plano.
.docx	No	Conviértelo a PDF antes de colocarlo.
.png y .jpg	No	Los documentos escaneados no se interpretan; usa PDF con texto.

Cuándo volver a ejecutar /setup: tras añadir una exportación nueva de LinkedIn, cartas de recomendación, resultados de candidaturas terminadas o una versión actualizada de tu CV.

6. Uso diario

1. Buscar ofertas

```
/scrape
```

Busca en los portales con tus consultas, elimina duplicados y presenta las ofertas ordenadas por encaje. Si salen demasiadas, puntúalas en lote primero:

```
/rank
```

Varios agentes en paralelo leen cada oferta y la puntúan en cinco dimensiones. Devuelve una lista ordenada con puntos fuertes y carencias de cada una. Los criterios excluyentes descartan la oferta, las fechas límite se marcan como urgentes y los anuncios caducados se señalan. Eliges un número y pasa directamente a /apply.

2. Solicitar una oferta

```
/apply https://jobindex.dk/job/1234567

# Si el portal bloquea el acceso automático, pega el texto de la oferta:
/apply <pega aquí la descripción completa del puesto>
```

El flujo completo de /apply es este:

- Lee la oferta y evalúa el encaje con tu perfil; te pregunta si quieres continuar.
- Redacta un CV y una carta adaptados, en LaTeX.
- Lanza un segundo agente, con contexto limpio, que investiga la empresa y critica los borradores.

- Revisa según esa crítica.
- Compila los PDF (lualatex para el CV, xelatex para la carta), los inspecciona y corrige la maquetación hasta que el CV ocupa exactamente 2 páginas y la carta 1, con la firma visible.
- Comprueba el CV como lo vería un ATS: contacto, orden de lectura y cobertura de palabras clave. Las palabras que tu perfil no respalda se señalan como carencia y nunca se añaden por rellenar.
- Presenta el resultado con una lista de verificación.

Todo lo que aparece en el CV y la carta se contrasta con tu perfil real: el sistema no inventa habilidades ni experiencia.

3. Compilar a mano los documentos

Normalmente /apply ya compila. Si necesitas repetirlo tras editar un `.tex`:

```
# Bash / zsh / Git Bash
cd cv && lualatex main_<empresa>_<puesto>.tex && cd ..
cd cover_letters && xelatex cover_<empresa>_<puesto>.tex && cd ..
```

```
# PowerShell
Set-Location cv; lualatex main_<empresa>_<puesto>.tex; Set-Location ..
Set-Location cover_letters; xelatex cover_<empresa>_<puesto>.tex; Set-Location ..
```

4. Preparar la entrevista y registrar resultados

```
/interview
/outcome
/outcome followup
```

- `/interview` crea un paquete de preparación para la etapa: la oferta exacta, el CV y la carta que leyó el entrevistador, feedback de rondas anteriores, investigación de la empresa y de los entrevistadores, preguntas probables asociadas a tus ejemplos STAR y simulacro opcional. Si hay un hueco en tu experiencia, propone respuestas honestas en vez de inventar.
- `/outcome` anota entrevistas, ofertas, rechazos o silencio; archiva el CV, la carta y la oferta en `documents/applications/` y actualiza el seguimiento.
- `/outcome followup` detecta candidaturas sin respuesta (por defecto, 10 días) y redacta un seguimiento breve con tu estilo. Solo redacta, nunca envía, y como máximo dos por candidatura. Tras registrar una etapa de entrevista, ofrece una nota de agradecimiento.

5. Otros comandos

Comando	Para qué sirve
`/gmail-sync`	Lee tu Gmail (con el conector de Gmail) y propone cambios de estado: invitaciones a entrevista, pruebas, ofertas, rechazos. Tú apruebas el lote antes de que se escriba nada. Las ofertas aceptadas o rechazadas las decides tú.
`/notion-sync`	Publica una vista de solo lectura del proceso en una base de datos de Notion mediante el servidor MCP oficial. Los ficheros del repo siguen siendo la fuente de verdad.
`/expand`	Enriquece tu perfil con fuentes públicas que ya enlazaste (GitHub, portfolio, Kaggle, Google Scholar) y programas de cursos.
`/upskill`	Analiza la brecha entre tu perfil y las ofertas guardadas, y genera un plan de estudio. También acepta una oferta: `/upskill <URL>`
`/html-report`	Genera un panel HTML sin dependencias a partir del CSV de seguimiento. Funciona sin conexión.
`/add-template`	Registra tu propia plantilla de CV o carta (LaTeX, Typst u otra). Opciones: `--list`, `--use <n>`, `--use default`.
`/add-portal`	Genera el buscador de un portal de empleo de tu mercado y lo prueba con una consulta real antes de registrarlo.

Comando	Para qué sirve
`/reset`	Borra datos. Admite `profile`, `documents` y `all`. Muestra qué se borrará y exige escribir RESET.

Comparativa salarial (opcional)

Si tienes datos de un sindicato, una encuesta o Glassdoor, puedes dárselos a /apply para que compare. Si no, ese paso se omite sin errores.

```
pip install openpyxl
python3 tools/convert_salary_excel.py ruta/a/datos-salariales.xlsx --source "Mis datos 2026"
```

Esto crea `salary_data.json` en la raíz. También puedes crearlo a mano; el formato está en `tools/README_SALARY_TOOL.md`.

7. Mapa de carpetas del repositorio

Ruta	Para qué sirve	¿La tocas?
`CLAUDE.md`	Tu perfil principal y las reglas del flujo.	Lo rellena /setup; puedes editarlo.
`.claude/commands/`	Definición de cada comando (apply, setup, rank, outcome...).	No, salvo que quieras cambiar el flujo.
`.claude/skills/job-application-assistant/`	Los ficheros 01 a 07: perfil, estilo de escritura, evaluación, plantillas, entrevistas.	Los rellena /setup; puedes ajustarlos.
`.claude/skills/job-scraper/`	Orquesta las búsquedas de /scrape.	No.
`.claude/skills/upskill/`	Análisis de brechas y plan de estudio.	No.
`.claude/settings.json`	Permisos compartidos de Claude Code, limitados a los buscadores y a la herramienta de salarios.	No.
`.agents/skills/`	Un buscador por portal, cada uno con su CLI.	Solo para añadir o copiar portales.
`cv/`	Plantilla moderncv (`main_example.tex`) y los CV generados por /apply.	Revisas los resultados.
`cover_letters/`	Clase `cover.cls`, ejemplo de carta y carpeta `OpenFonts/` con las fuentes Lato y Raleway.	Revisas los resultados.
`templates/`	Plantillas propias registradas con /add-template.	Vía /add-template.
`documents/`	Tu material de partida y el archivo de candidaturas. Excluida de Git.	Sí. Es tu carpeta principal.
`company_research/`	Por el nombre, investigación sobre empresas; el README no detalla su contenido.	Normalmente no.
`job_scraper/`	Estado de las búsquedas: ofertas ya vistas y resultados.	No.
`gmail_sync/`	Estado de /gmail-sync: mensajes procesados y última sincronización.	No.
`upskill/`	Informes en markdown de cada ejecución de /upskill.	Los lees.
`job_search_tracker.csv`	Hoja de seguimiento de todas tus candidaturas. Excluida de Git.	La mantiene /outcome.
`salary_lookup.py`	Herramienta de comparativa salarial (con tus propios datos).	No.
`tools/`	Scripts de apoyo: conversión de salarios, comprobación de PDF, triaje de actualizaciones, linters de CI.	Algunos los ejecutas tú.

Ruta	Para qué sirve	¿La tocas?
`tests/`	Pruebas automáticas del proyecto.	No.
`assets/mascot/`	Imágenes de la mascota (el pájaro mensajero Pip).	No.
`SETUP.md`, `CHANGELOG.md`, `CONTRIBUTING.md`, `SECURITY.md`, `AGENTS.md`	Guía de instalación, historial de versiones, normas para contribuir, política de seguridad y guía para otros agentes (Codex, Gemini CLI).	Léelos.

El proyecto excluye de Git el seguimiento, los datos salariales, `documents/` y los archivos de candidaturas, así que no entran en el historial aunque hagas commit.

8. Actualizar sin perder tus datos

El proyecto evoluciona rápido. Actualiza a una versión etiquetada (release) en vez de a master: cada una es un punto de control descrito en CHANGELOG.md. Antes confirma tus cambios personales con commit en tu repositorio privado; un árbol de trabajo sin confirmar es la causa más habitual de que git pull se niegue a fusionar.

```
git remote add upstream https://github.com/MadsLorentzen/ai-job-search.git   # solo la primera vez
git fetch upstream --tags
python3 tools/check_upstream_updates.py   # qué ficheros personalizados toca la actualización
python3 tools/upstream_triage.py --remote upstream   # qué commits merecen revisión
git merge v1.0.0                          # o: git merge upstream/master
```

Sustituye `v1.0.0` por la etiqueta real que quieras. Un conflicto en un fichero personalizado significa que el proyecto cambió la metodología en una sección que tú habías adaptado: conserva tus datos y adopta el cambio de método a su alrededor.

9. Problemas frecuentes

Síntoma	Solución
salary_data.json not found	Es normal si no configuraste salarios. /apply omite ese paso.
Los buscadores no funcionan	Comprueba que Bun está instalado y que ejecutaste `bun install` en cada carpeta cli. Necesitan conexión a internet.
Errores al compilar el CV	Usa lualatex, no pdflatex. Verifica que tu distribución incluye el paquete moderncv.
Faltan fuentes en la carta	Revisa que existe `cover_letters/OpenFonts/fonts/` con los ficheros de Lato y Raleway.
MiKTeX se bloquea al pedir paquetes	Activa la instalación silenciosa con los dos comandos `initexmf` del paso 4.
Permisos antiguos de una versión previa	Si clonaste hace tiempo, borra `.claude/settings.local.json`: `rm .claude/settings.local.json`
git pull se niega a fusionar	Tienes cambios sin confirmar: haz commit de tu personalización y repite.

10. Seguridad y límites que conviene conocer

- **Tus datos son sensibles.** El perfil contiene contacto, historial y expectativas salariales. Usa un repositorio privado.
- **Las ofertas son entrada no fiable.** El flujo no sigue instrucciones incrustadas en ellas ni abre enlaces de su cuerpo, pero esa defensa se basa en instrucciones, no en un aislamiento técnico. En portales desconocidos, revisa lo que se descargó y lo que se escribió antes de enviar nada.
- **LinkedIn.** linkedin-search usa endpoints públicos y es solo para uso personal: el acceso automatizado va contra sus condiciones, así que mantén un volumen bajo.

- **Código de otros forks.** Los buscadores se ejecutan con permisos preaprobados en tu máquina. Si copias uno de otro fork, lee todo el código y ejecuta sus pruebas sin conexión antes de usarlo.
- **Tú pulsas Enviar.** El sistema prepara documentos y borradores; no envía candidaturas ni correos por ti.
- **Coste.** El flujo con dos agentes y la verificación de PDF consume bastantes tokens. Tenlo en cuenta con una suscripción con límites de uso.
- **Estafas.** El proyecto no tiene ninguna criptomoneda, token ni programa de patrocinio asociado. Cualquier cosa que diga lo contrario no es oficial.